Лабораторна робота № 2

Оптимізаційна задача та аналіз чутливості

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи · **Максимальна оцінка:** 4 бали

Мета роботи

Побудувати оптимізаційну модель у двох середовищах, прочитати звіт про стійкість і **ухвалити на його основі господарське рішення**.

Розв'язати задачу — не мета. Solver зробить це за дві секунди. Робота оцінюється за те, що ви зумієте зі звіту витягнути.

Дві частини

Частина	Задача	Де виконується
1	виробнича задача — Excel і Python	аудиторно
2	транспортна задача — Python	самостійно

Друга частина потрібна не для повторення. Там та сама методика поводиться інакше, і ви маєте побачити, де саме проходить межа її застосовності.

Що здається

1. Цей файл `.ipynb` із заповненими розв'язками. **Виконується згори донизу на чистому ядрі** (Kernel → Restart & Run All) без помилок — перевіряється першою дією на захисті.
2. Файл Excel з моделлю, налаштуваннями Solver і **звітом про стійкість** окремим аркушем.
3. Заповнена декларація про використання ШІ (етап 12).

Оцінювання

Бали	Що досягнуто
0	Модель не сформульовано або розв'язок не отримано
1	Розв'язок отримано лише в одному середовищі, звіт про стійкість не проаналізовано
2	Задачу розв'язано в обох середовищах і звіти зіставлено, але тіньові ціни та діапазони не інтерпретовано
4	Звіт прочитано й перевірено експериментом, ухвалено обґрунтоване рішення про закупівлю, виконано частину 2 з поясненням розбіжності

Крок 0. Ваші дані та варіант

In [99]:
```
ПІБ = 'Колбасюк Данило Іванович'        # напр. 'Шевченко Тарас'
ГРУПА = 'ШІ-32'      # напр. 'КН-31'
ВАРІАНТ = 6     # ваш номер за списком групи

assert ПІБ and ГРУПА and 1 <= ВАРІАНТ <= 30, 'Заповніть ПІБ, ГРУПУ та ВАР
```

In [100…
```
# --- клітинку не редагувати: вона видає дані вашого варіанта -----------
import json, numpy as np, pandas as pd

ДАНІ_ВАРІАНТІВ = json.loads('''{"1": {"фірма": "«Астро-ТВ»", "що": "телев

V = ДАНІ_ВАРІАНТІВ[str(ВАРІАНТ)]
A = np.array(V['A'], float)          # витрати ресурсу і на один виріб j
c = np.array(V['c'], float)          # прибуток з одного виробу
b = np.array(V['b'], float)          # запас ресурсу на плановий період
вироби, ресурси = V['вироби'], V['ресурси']

print('ВАРІАНТ %d · %s, %s, %s' % (ВАРІАНТ, ПІБ, ГРУПА, V['фірма']))
print('=' * 78)
print('Підприємство %s виготовляє %s. Одиниця виміру плану: %s.' % (V['фі
print()
таб = pd.DataFrame(A, index=ресурси, columns=вироби).astype(int)
таб['ЗАПАС'] = b.astype(int)
print(таб.to_string())
print()
print(pd.DataFrame([c.astype(int)], index=['Прибуток з одиниці'], columns
print()
print('ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА (знадобиться на етапі 6)')
print('  ресурс: %s' % ресурси[V['ресурс_пропозиції']])
print('  ціна:   %.1f за одиницю' % V['ціна_пропозиції'])
print('  обсяг:  до %d одиниць' % V['обсяг_пропозиції'])
```

```
ВАРІАНТ 6 · Колбасюк Данило Іванович, ШІ-32, «Longer Boats»
==============================================================================
====
Підприємство «Longer Boats» виготовляє яхти. Одиниця виміру плану: шт./міс
яць.

                      Sting  Ray  Breaker  Marlin  ЗАПАС
Склопластик, м²           3    2        1       4    630
Столярний цех, год        4    2        2       1    391
Фарбувальний цех, год     3    5        1       2   1349
Такелаж, компл.           3    2        0       2    811
Ділянка добудови, год     6    4        6       3   1085

                    Sting  Ray  Breaker  Marlin
Прибуток з одиниці     40   62       19      57

ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА (знадобиться на етапі 6)
  ресурс: Столярний цех, год
  ціна:   29.5 за одиницю
  обсяг:  до 268 одиниць
```

In [101…
```
import matplotlib.pyplot as plt
```

```
try:
    import pulp
    ПУЛП = True
except ImportError:
    ПУЛП = False
    print('pulp не встановлено — скористайтеся scipy.optimize.linprog')
from scipy.optimize import linprog

pd.set_option('display.width', 180)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                     'axes.spines.right': False, 'font.size': 9})
```

ЧАСТИНА 1. Виробнича задача

Етап 1. Формалізація

Перш ніж будувати таблицю, запишіть модель словами й формулами. Це найважливіші двадцять хвилин роботи: помилку тут не виправить жодна кнопка.

Завдання 1.1

Заповніть у клітинці нижче:

- **змінні рішення** — що саме ви обираєте і в яких одиницях;
- **цільова функція** — що робите найбільшим, з коефіцієнтами вашого варіанта;
- **обмеження** — по одному рядку на кожен ресурс, з числами вашого варіанта;
- **умови невід'ємності**.

Пишіть формулами, а не описом. Приклад запису одного обмеження: `8·x1 + 4·x2 + 0·x3 + 2·x4 ≤ 1280   (конвеєр A)`

Відповідь 1.1 — символічна модель

Змінні рішення: x1, x2, x3, x4 - кількості різних яхт, шт./місяць

Цільова функція: 40*x1+62*x2+19*x3+57*x4 -> max

Обмеження: 3*x1+2*x2+1*x3+4*x4<=630; 4*x1+2*x2+2*x3+1*x4<=391; 3*x1+5*x2+1*x3+2*x4<=1349; 3*x1+2*x2+0*x3+2*x4<=811; 6*x1+4*x2+6*x3+3*x4<=1085

Невід'ємність: x1,x2,x3,x4>=0

Завдання 1.2 — питання на розуміння

Чому запас ресурсу — це **параметр**, а не змінна рішення? Що змінилося б у моделі, якби підприємство могло докупити ресурс у будь-якій кількості?

Відповідь 1.2: Запас ресурсу стає ще одною змінною в моделі, яку б треба було врахувати для обчислення максимального прибутку.

Етап 2. Розв'язання в Excel

1. Побудуйте табличну модель: окремо клітинки змінних рішення, окремо витрати ресурсів, окремо цільова функція. Не змішуйте вхідні дані з формулами.
2. Налаштуйте **Пошук рішення (Solver)**: цільова клітинка на максимум, клітинки змінних, обмеження по кожному ресурсу, невід'ємність, метод «Симплекс-метод для лінійних задач».
3. Отримайте розв'язок і **обов'язково замовте «Звіт про стійкість»** окремим аркушем.

Що подати: файл Excel зі скріншотом налаштувань Solver, оптимальним планом і звітом.

Завдання 2.1

Перенесіть результат Excel у клітинку нижче — його порівняємо з Python.

In [102…
```
# впишіть числа, які дав Excel
EXCEL_ПЛАН    = [0, 155.67, 0, 79.67]     # оптимальний випуск кожного ви
EXCEL_ПРИБУТОК = 14192.33                 # значення цільової функції

print('Excel:', EXCEL_ПЛАН, '| прибуток', EXCEL_ПРИБУТОК)
```

```
Excel: [0, 155.67, 0, 79.67] | прибуток 14192.33
```

Етап 3. Розв'язання в Python

Завдання 3.1

Побудуйте ту саму модель у Python і розв'яжіть її. Можна `pulp`, можна `scipy.optimize.linprog` — на ваш вибір.

Очікуваний результат: оптимальний план і значення цільової функції.

Увага на знак. `linprog` завжди мінімізує. Щоб максимізувати прибуток, передають `-c`, а результат беруть як `-r.fun`.

In [103…
```
prob=pulp.LpProblem("Максимізація прибутку", pulp.LpMaximize)
x1=prob.add_variable("x1", lowBound=0)
x2=prob.add_variable("x2", lowBound=0)
x3=prob.add_variable("x3", lowBound=0)
x4=prob.add_variable("x4", lowBound=0)
```

```
prob+=40*x1+62*x2+19*x3+57*x4 # цільова
prob+=(3*x1+2*x2+1*x3+4*x4<=630, "c1") # обмеження для склопластику
prob+=(4*x1+2*x2+2*x3+1*x4<=391, "c2") # обмеження для столярного цеху
prob+=(3*x1+5*x2+1*x3+2*x4<=1349, "c3") # обмеження для фарбувального цех
prob+=(3*x1+2*x2+0*x3+2*x4<=811, "c4") # обмеження для такелажу
prob+=(6*x1+4*x2+6*x3+3*x4<=1085, "c5") # обмеження для ділянки добудови
prob.solve()
план=[pulp.value(x1), pulp.value(x2), pulp.value(x3), pulp.value(x4)]
print('План: ', план)
прибуток=pulp.value(prob.objective)
print('Прибуток: ', прибуток)
```

```
План:  [0.0, 155.66667, 0.0, 79.666667]
Прибуток:  14192.333559000002
c:\Users\JustDector\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packa
ges\pulp\pulp.py:1706: UserWarning: Spaces are not permitted in the name.
Converted to '_'
  warnings.warn("Spaces are not permitted in the name. Converted to '_'")
```

Самоперевірка етапу 3. Excel і Python мають дати той самий результат.

In [104…

```
assert план is not None and прибуток is not None, 'спершу розв'яжіть зада
assert abs(прибуток - EXCEL_ПРИБУТОК) < 0.5, (
    'Excel дав %.2f, Python %.2f — знайдіть, де розбіжність' % (EXCEL_ПРИ
assert np.allclose(np.array(план, float), np.array(EXCEL_ПЛАН, float), at
    'плани не збігаються — перевірте обмеження'
print('Етап 3 пройдено: обидва середовища дали однаковий результат')
```

```
Етап 3 пройдено: обидва середовища дали однаковий результат
```

Етап 4. Читання звіту про стійкість

Звіт Excel має два блоки. Заповніть обидві таблиці — беріть числа зі **звіту**, а не з власних обчислень.

Завдання 4.1 — блок «Обмеження»

Ресурс	Кінцеве значення	Тіньова ціна	Права частина	Допустиме збільшення	Допустиме зменшення
Склопластик, м2	630	8,666666667	690	670	239
Столярний цех, год	391	22,33333333	391	134	233,5
Фарбувальний цех, год	937,6666667	0	1349	1E+30	411,3333333
Такелаж, компл.	470,6666667	0	811	1E+30	340,3333333
Ділянка добудови, год	861,6666667	0	1085	1E+30	223,3333333

Які ресурси в дефіциті, а які в надлишку? Як ви це визначили?

Всі, крім столярного цеху - в надлишку. Столярний цех - дефіцит. Для цього порівнюємо кінцеве значення та праву частину.

Завдання 4.2 — блок «Змінні»

Виріб	Кінцеве значення	Зведена вартість	Цільовий коефіцієнт	Допустиме збільшення	Допустиме зменшення
Sting	0	-75,33333333	40	75,33333333	1E+30
Ray	155,6666667	0	62	52	29,42857143
Breaker	0	-34,33333333	19	34,33333333	1E+30
Marlin	79,66666667	0	57	67	26

У вашому варіанті принаймні один виріб не виробляється зовсім. Який саме, і на скільки мусив би зрости прибуток з його одиниці, щоб він увійшов у план? Звідки ви взяли це число? Повинен збільшитись прибуток Breaker, найменш допустиме збільшення - 34.33333 (зведена вартість) + цільовий коефіцієнт (19) = 53.3333333

Етап 5. Перевірка звіту експериментом

Звіт стверджує, що тіньова ціна дорівнює певному числу і чинна в певних межах. Excel просто повідомляє ці числа. Python дозволяє їх **перевірити**.

Завдання 5.1

Візьміть найдорожчий за тіньовою ціною ресурс. Збільште його запас **на одну одиницю**, розв'яжіть задачу заново і порівняйте приріст прибутку з тіньовою ціною зі звіту.

In [105…
```
prob.constraints["c2"]+=-1
prob.solve()
план=[pulp.value(x1), pulp.value(x2), pulp.value(x3), pulp.value(x4)]
print('План: ', план)
прибуток2=pulp.value(prob.objective)
print('Прибуток: ', прибуток2)
```

```
План:  [0.0, 156.33333, 0.0, 79.333333]
Прибуток:  14214.666440999998
```

Відповідь 5.1: *(приріст прибутку = ..., тіньова ціна зі звіту = ..., збігаються?)*
Приріст прибутку - 22,333, тіньова ціна зі звіту - 22,333, збігаються

Завдання 5.2

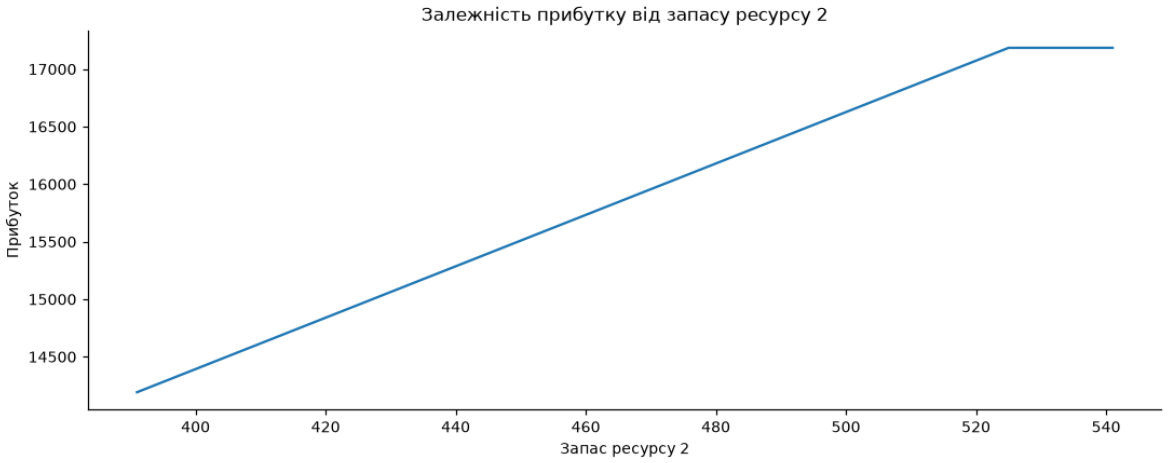
Тепер знайдіть **межу діапазону**. Прогоніть задачу для запасу цього ресурсу від базового значення і далі вгору з кроком 1, побудуйте графік прибутку і графік його приросту (нахилу).

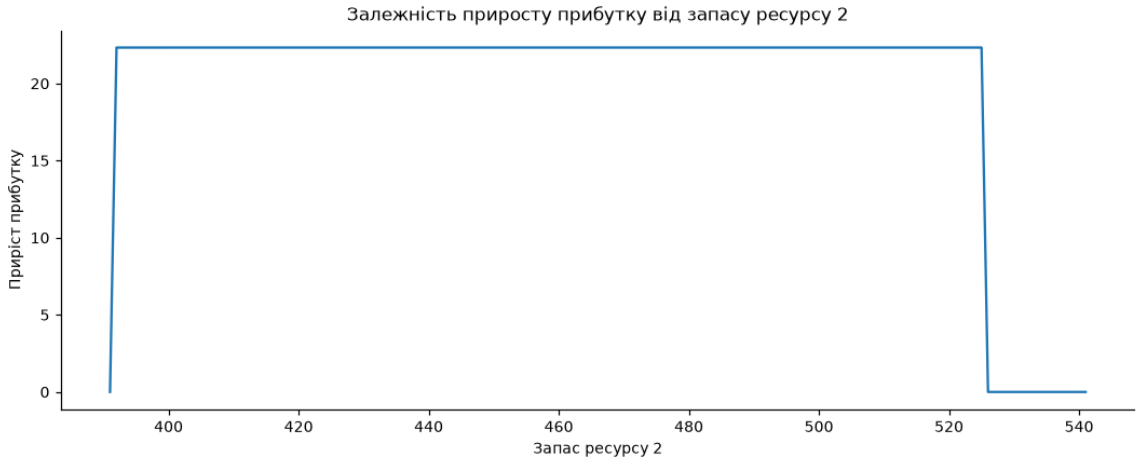
Очікуваний результат: два графіки один під одним. На нижньому має бути видно горизонтальну ділянку на рівні тіньової ціни, а потім злам.

In [106…

```
prob.constraints["c2"]=4*x1+2*x2+2*x3+1*x4<=391
prob.solve()
constraint_x=[prob.constraints["c2"].constant*-1]
profit=[]
profit_increase=[0]
profit.append(pulp.value(prob.objective))
res=0
for i in range(150):
    prob.constraints["c2"]+=-1
    prob.solve()
    constraint_x.append(prob.constraints["c2"].constant*-1)
    profit.append(pulp.value(prob.objective))
    profit_increase.append(profit[-1]-profit[-2])
    if(profit_increase[-1]>0):
        res=constraint_x[-1]
print(res-constraint_x[0])
fig,axes=plt.subplots(2,1,figsize=(10,8))
axes[0].plot(constraint_x, profit)
axes[0].set_title('Залежність прибутку від запасу ресурсу 2')
axes[0].set_xlabel('Запас ресурсу 2')
axes[0].set_ylabel('Прибуток')
axes[1].plot(constraint_x, profit_increase)
axes[1].set_title('Залежність приросту прибутку від запасу ресурсу 2')
axes[1].set_xlabel('Запас ресурсу 2')
axes[1].set_ylabel('Приріст прибутку')
plt.tight_layout()
plt.show()
```

```
134
```





Відповідь 5.2: *(на скільки одиниць можна збільшити запас, доки тіньова ціна не зміниться; чи збігається знайдена межа з «допустимим збільшенням» зі звіту Excel)* 134, число збігається зі звітом

Етап 6. Рішення про закупівлю

Постачальник пропонує вам додатковий обсяг ресурсу — див. блок «ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА» у ваших даних.

Завдання 6.1

Дайте відповідь директорові на три питання, спираючись на числа, а не на здоровий глузд:

1. **Брати чи не брати?** Порівняйте ціну пропозиції з тіньовою ціною.
2. **Скільки саме брати?** Постачальник пропонує більше, ніж вам потрібно.
3. **Скільки на цьому заробимо?** Порахуйте виграш і перевірте його прямим розв'язанням задачі з новим запасом.

In [107…

```
prob.constraints["c2"]=4*x1+2*x2+2*x3+1*x4<=391
prob.solve()
old=pulp.value(prob.objective)
print(old)
```

```
prob.constraints["c2"]+=-268 # Після 134 прибуток збільшуватись не буде
prob.solve()
new=pulp.value(prob.objective)
print(new)
cost1=134*29.5
gain1=new-old-cost1
cost2=cost1*2
gain2=new-old-cost2
print(gain1)
print(gain2)
```

```
14192.333559000002
17185.0
-960.3335590000024
-4913.333559000002
```

Відповідь 6.1:

- Брати чи ні і чому: Вказана ціна - 29.5, а тіньова - 22.333, тому брати не вигідно.
- Скільки одиниць має сенс купити і чому не більше: Навіть якби вказана ціна була б меншою, ніж тіньова, брати більше, ніж 134 одиниці, не потрібно. Не змінюючи ціни, логічно, потрібно взяти 0 одиниць.
- Очікуваний виграш: Тут виграшів не буде, тільки втрати. За одиницю втрата приблизно 7.1667, а якщо взяти 134 години: 134*22.333-134*29.5=-960,38, а всі 268 годин - -4913.333
- Перевірка прямим розв'язанням дала: -960.33 при 134 годинах, -4913.333 при максимальній кількості

Етап 7. Перевірка розв'язку

Завдання 7.1

Порахуйте дві величини:

- скільки виробів мають **ненульовий** випуск;
- скільки обмежень **зв'язні** (ресурс витрачено повністю).

Порівняйте.

In [108…
```
prob.constraints["c2"]=4*x1+2*x2+2*x3+1*x4<=391
prob.solve()
ненульових = 0
звязних = 0
план=[pulp.value(x1), pulp.value(x2), pulp.value(x3), pulp.value(x4)]
for element in план:
    if element>0:
        ненульових+=1
for element in prob.constraints.values():
    if element.slack==0:
        звязних+=1
```

In [109…
```
assert ненульових is not None and звязних is not None, 'спершу порахуйте'
```

```
print('ненульових змінних: %s | зв'язних обмежень: %s' % (ненульових, звя
assert ненульових == звязних, ('числа не збіглися — це сигнал. Перевірте
                                'і поясніть причину в клітинці нижче')
print('Етап 7 пройдено')
```

```
ненульових змінних: 2 | зв'язних обмежень: 2
Етап 7 пройдено
```

Відповідь 7.1: Числа збіглися, якби вони не збіглися, то це не була б помилка; просто більше виробів можуть використовувати ті самі ресурси, або навпаки, для виробів використовується більше різних ресурсів

Етап 8. Цілочисельність

Ваш оптимальний план містить дробові числа. Виготовити 63,3 виробу неможливо.

Завдання 8.1

Порівняйте три варіанти:

1. розв'язок без умови цілочисельності (уже маєте);
2. **округлений** план — округліть кожне значення до цілого й перевірте, чи він узагалі допустимий і скільки прибутку дає;
3. справжній **цілочисельний** оптимум — розв'яжіть задачу з умовою, що всі змінні цілі (`cat='Integer'` у PuLP або `integrality=1` у `linprog`).

In [110…
```
# ВАШ КОД
prob=pulp.LpProblem("Максимізація прибутку", pulp.LpMaximize)
x1=prob.add_variable("x1", lowBound=0, cat='Integer')
x2=prob.add_variable("x2", lowBound=0, cat='Integer')
x3=prob.add_variable("x3", lowBound=0, cat='Integer')
x4=prob.add_variable("x4", lowBound=0, cat='Integer')
prob+=40*x1+62*x2+19*x3+57*x4 # цільова
prob+=(3*x1+2*x2+1*x3+4*x4<=630, "c1") # обмеження для склопластику
prob+=(4*x1+2*x2+2*x3+1*x4<=391, "c2") # обмеження для столярного цеху
prob+=(3*x1+5*x2+1*x3+2*x4<=1349, "c3") # обмеження для фарбувального цех
prob+=(3*x1+2*x2+0*x3+2*x4<=811, "c4") # обмеження для такелажу
prob+=(6*x1+4*x2+6*x3+3*x4<=1085, "c5") # обмеження для ділянки добудови
prob.solve()
план=[pulp.value(x1), pulp.value(x2), pulp.value(x3), pulp.value(x4)]
print('План: ', план)
прибуток3=pulp.value(prob.objective)
print('Прибуток: ', прибуток3)
```

```
План:  [0.0, 156.0, 0.0, 79.0]
Прибуток:  14175.0
```

Відповідь 8.1:

Варіант	План	Прибуток	Допустимий?
без умови цілочисельності	[0.0, 155.66667, 0.0, 79.666667]	14192.333559000002	ні, дробові числа

Варіант	План	Прибуток	Допустимий?
округлений	[0.0, 156.0, 0.0, 80.0]	14232.0	ні, не входить в обмеження
цілочисельний оптимум	[0.0, 156.0, 0.0, 79.0]	14175.0	так

Висновок: *(чи можна округлювати і чому)* В цьому випадку ні, бо це порушує задані обмеження.

ЧАСТИНА 2. Транспортна задача

Друга частина виконується самостійно. Тут ви застосуєте ту саму методику до задачі іншого класу — і побачите, що вона поводиться інакше.

Етап 9. Ваші дані

Завдання 9.1

Сформуйте власні дані:

- **три міста-постачальники** — за першими трьома літерами вашого **прізвища**;
- **чотири міста-споживачі** — за першими чотирма літерами вашого **імені**;
- **матриця відстаней 3 × 4** — реальні відстані автошляхами за Google Maps, у кілометрах;
- **запаси і попит** — невеликі цілі числа, підберіть так, щоб **сумарний запас дорівнював сумарному попиту**.

> Якщо на якусь літеру міста немає — беріть наступну літеру прізвища чи імені. Якщо сумарний запас не дорівнює попиту, задача з обмеженнями-рівностями **не матиме розв'язку взагалі**. Це не помилка Python — це властивість моделі.

In [111…

```
# ВАШ КОД — впишіть свої дані
постачальники = ['Київ', 'Одеса', 'Львів']            # напр. ['Шостка',
споживачі     = ['Дніпро','Авдіївка','Нікополь','Луцьк']            # нап
D = np.array([[443,663,497,412],
              [448,665,367,697],
              [921,1150,898,150]])             # матриця відстаней 3 x 4
запас = [137, 142, 169]                    # 3 числа
попит = [112, 86, 142, 108]                    # 4 числа

assert len(постачальники) == 3 and len(споживачі) == 4, 'потрібно 3 поста
assert D.shape == (3, 4), 'матриця відстаней має бути 3 x 4'
assert sum(запас) == sum(попит), 'сумарний запас має дорівнювати сумарном
pd.DataFrame(D, index=постачальники, columns=споживачі)
```

Out[111…

	Дніпро	Авдіївка	Нікополь	Луцьк
Київ	443	663	497	412
Одеса	448	665	367	697
Львів	921	1150	898	150

Етап 10. Розв'язання транспортної задачі

Завдання 10.1

Розв'яжіть задачу **двічі різними засобами**: наприклад `pulp` і `scipy.optimize.linprog(method='highs')`. Обмеження — рівності.

Для кожного розв'язку виведіть: оптимальний план перевезень, сумарну вартість і **тіньові ціни всіх обмежень**.

In [112…

```
# pulp
prob=pulp.LpProblem("Транспортна задача", pulp.LpMinimize)
x = pulp.LpVariable.dicts("x", (постачальники, споживачі), lowBound=0, ca
prob+=pulp.lpSum(D[i,j] * x[постачальники[i]][споживачі[j]] for i in rang
for i in range(len(постачальники)):
    prob+=(pulp.lpSum(x[постачальники[i]][j] for j in споживачі) == запас
for j in range(len(споживачі)):
    prob+=(pulp.lpSum(x[i][споживачі[j]] for i in постачальники) == попит
prob.solve()
print(pulp.value(prob.objective))
for i in постачальники:
    for j in споживачі:
        if x[i][j].value() > 0:
            print(f"{i} -> {j}: {x[i][j].value()}")
for name, constraint in prob.constraints.items():
    print(name, constraint.pi)
```

```
204106.0
Київ -> Дніпро: 51.0
Київ -> Авдіївка: 86.0
Одеса -> Нікополь: 142.0
Львів -> Дніпро: 61.0
Львів -> Луцьк: 108.0
supply_Київ 420.0
supply_Одеса 367.0
supply_Львів 898.0
demand_Дніпро 23.0
demand_Авдіївка 243.0
demand_Нікополь 0.0
demand_Луцьк -748.0
```

```
c:\Users\JustDector\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packa
ges\pulp\pulp.py:1706: UserWarning: Spaces are not permitted in the name.
Converted to '_'
  warnings.warn("Spaces are not permitted in the name. Converted to '_'")
```

In [113…

```
# scipy
import scipy
m=len(постачальники)
n=len(споживачі)
c = D.flatten()
A_eq = []
b_eq = []
for i in range(m):
    row = np.zeros(m*n)
    row[i*n:(i+1)*n] = 1
    A_eq.append(row)
    b_eq.append(запас[i])
for j in range(n):
    row = np.zeros(m*n)
    row[j::n] = 1
    A_eq.append(row)
    b_eq.append(попит[j])
A_eq = np.array(A_eq)
b_eq = np.array(b_eq)
bounds=[(0, None)] * (m*n)
res = scipy.optimize.linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, meth
print(res.fun)
print(res.x.reshape((m,n)))
shadow_supply = res.eqlin.marginals[:len(постачальники)]
shadow_demand = res.eqlin.marginals[len(постачальники):]
print(shadow_supply)
print(shadow_demand)
```

```
204106.0
[[ 51.  86.   0.   0.]
 [  0.   0. 142.   0.]
 [ 61.   0.   0. 108.]]
[ -0.  -0. 478.]
[ 443.  663.  367. -328.]
```

Завдання 10.2 — головне питання цієї частини

Порівняйте два набори тіньових цін.

Сумарна вартість у двох розв'язувачів однакова. А тіньові ціни?

Якщо вони різні — це не помилка. Знайдіть закономірність: подивіться на різницю між відповідними значеннями окремо для постачальників і окремо для споживачів.

In [114…

```
# ВАШ КОД — порівняння
pulp_supply = np.array([
    prob.constraints[f"supply_{i}"].pi
    for i in постачальники
])

pulp_demand = np.array([
    prob.constraints[f"demand_{j}"].pi
    for j in споживачі
])

# Тіньові ціни SciPy
scipy_supply = res.eqlin.marginals[:len(постачальники)]
```

```
scipy_demand = res.eqlin.marginals[len(постачальники):]
diff_supply = pulp_supply - scipy_supply
diff_demand = pulp_demand - scipy_demand
print("Різниці для постачальників:")
for name, diff in zip(постачальники, diff_supply):
    print(f"{name}: {diff}")

print("\nРізниці для споживачів:")
for name, diff in zip(споживачі, diff_demand):
    print(f"{name}: {diff}")
```

```
Різниці для постачальників:
Київ: 420.0
Одеса: 367.0
Львів: 420.0

Різниці для споживачів:
Дніпро: -420.0
Авдіївка: -420.0
Нікополь: -367.0
Луцьк: -420.0
```

Відповідь 10.2:

- Вартість у двох розв'язувачів: однакова, 204106
- Тіньові ціни збіглися чи ні: не збіглись
- Яку закономірність ви побачили в різниці: зустрічається тільки два унікальних значення: 367 та 420
- Що це означає для того, хто збирається ухвалювати рішення за цим звітом: це не змінює суму ui+vi=c_ij і тому не змінює оптимальне рішення

Етап 11. Перевірка розв'язку — і чому вона тут не працює

Завдання 11.1

Порахуйте для транспортної задачі те саме, що на етапі 7:

- кількість **ненульових перевезень**;
- кількість **зв'язних обмежень**.

Порівняйте з числом `3 + 4 = 7` — кількістю обмежень у задачі.

In [115…

```
ненульові_перевезення = sum(
    1
    for i in постачальники
    for j in споживачі
    if x[i][j].value() > 1e-6
)

print("Кількість ненульових перевезень:", ненульові_перевезення)
```

```
звязні_обмеження = sum(
    1
    for constraint in prob.constraints.values()
    if abs(constraint.slack) < 1e-6
)

print("Кількість зв'язних обмежень:", звязні_обмеження)
```

```
Кількість ненульових перевезень: 5
Кількість зв'язних обмежень: 7
```

Відповідь 11.1:

- Ненульових перевезень: 5
- Зв'язних обмежень: 7
- Чому в транспортній задачі зв'язними виявляються **всі** обмеження: тому що задача збалансована
- На скільки ненульових перевезень менше і чому саме на стільки: на 2 менше, додатніх перевезень менше, ніж елементів базисного плану

Завдання 11.2 — візуалізація

Побудуйте дві теплові карти поруч: матрицю відстаней і матрицю оптимальних перевезень. Підпишіть осі та значення в комірках.

За бажанням (не оцінюється): граф маршрутів через `networkx`*, де товщина ребра пропорційна обсягу перевезення.*

In [116…

```
import seaborn as sns

# Матриця оптимальних перевезень з PuLP
X = np.array([
    [x[i][j].value() for j in споживачі]
    for i in постачальники
])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Матриця відстаней
sns.heatmap(
    D,
    annot=True,
    fmt=".0f",
    cmap="Blues",
    xticklabels=споживачі,
    yticklabels=постачальники,
    ax=axes[0]
)

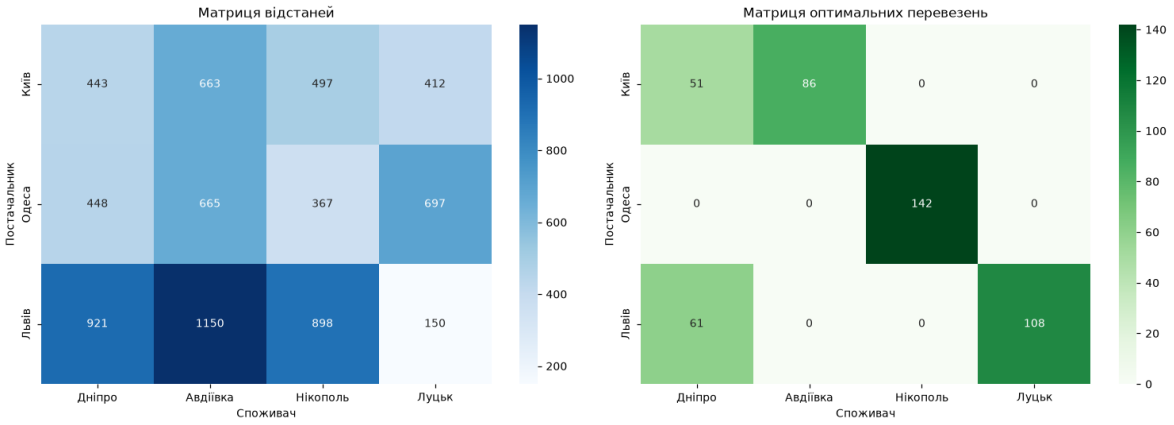
axes[0].set_title("Матриця відстаней")
axes[0].set_xlabel("Споживач")
axes[0].set_ylabel("Постачальник")

# 2. Матриця оптимальних перевезень
sns.heatmap(
    X,
```

```
    annot=True,
    fmt=".0f",
    cmap="Greens",
    xticklabels=споживачі,
    yticklabels=постачальники,
    ax=axes[1]
)

axes[1].set_title("Матриця оптимальних перевезень")
axes[1].set_xlabel("Споживач")
axes[1].set_ylabel("Постачальник")

plt.tight_layout()
plt.show()
```



Висновок: *(одне речення про те, що видно на теплових картах)* Завжди вибирається найменша відстань.

Етап 12. Декларація про використання інструментів штучного інтелекту

Користуватися мовними моделями **дозволено**. Обов'язковою є прозорість. Порушенням є не використання інструмента, а нездатність пояснити власний код і власні числа на захисті.

Для кожного етапу оберіть одне: `'ні'`, `'синтаксис'`, `'код, перевірено'`, `'текст, перероблено'`, `'текст, як є'`.

In [117…

```
ІНСТРУМЕНТИ_ШІ = 'ChatGPT'     # напр. 'ChatGPT' або 'не використовував'

ДЕКЛАРАЦІЯ = {
    'етап 1, формалізація моделі':          'ні',
    'етап 2, побудова моделі в Excel':      'ні',
    'етап 3, код на Python':                'ні',
    'етапи 4–5, читання та перевірка звіту': 'ні',
    'етап 6, рішення про закупівлю':        'ні',
    'етапи 7–8, перевірка та цілочисельність': 'код, перевірено',
    'етапи 9–11, транспортна задача':       'текст, перероблено',
}
```

```
# Що саме ви перевірили за згенерованим кодом чи текстом — конкретно.
ЩО_ПЕРЕВІРЕНО = "Розв'язки транспортної задачі, знаходження закономірност

# True означає: я можу пояснити кожен рядок свого коду, відтворити будь-я
# цієї роботи та внести зміни в модель на вимогу під час захисту.
ПІДТВЕРДЖУЮ = True
```

In [118…

```
# --- перевірка повноти декларації: клітинку не редагувати --------------
ДОЗВОЛЕНІ = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено',
assert ІНСТРУМЕНТИ_ШІ.strip(), 'Вкажіть інструменти або напишіть «не вико
порожні = [k for k, v in ДЕКЛАРАЦІЯ.items() if not str(v).strip()]
assert not порожні, 'Не заповнено: %s' % '; '.join(порожні)
чужі = {v for v in ДЕКЛАРАЦІЯ.values() if v not in ДОЗВОЛЕНІ}
assert not чужі, 'Недопустимі значення: %s' % ', '.join(sorted(чужі))
assert len(ЩО_ПЕРЕВІРЕНО.strip()) >= 120, ('Опишіть конкретніше (зараз %d
                                           'потрібно щонайменше 120)' % l
assert ПІДТВЕРДЖУЮ is True, 'Підтвердьте, що можете пояснити власний код
print('ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ')
print('=' * 78)
print('%-42s %s' % ('автор:', ПІБ))
print('%-42s %s' % ('інструменти:', ІНСТРУМЕНТИ_ШІ))
print('-' * 78)
for k, v in ДЕКЛАРАЦІЯ.items():
    print('%-42s %s' % (k + ':', v))
print('-' * 78)
print(ЩО_ПЕРЕВІРЕНО)
```

```
ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ
==============================================================================
====
автор:                                     Колбасюк Данило Іванович
інструменти:                               ChatGPT
------------------------------------------------------------------------------
----
етап 1, формалізація моделі:               ні
етап 2, побудова моделі в Excel:           ні
етап 3, код на Python:                     ні
етапи 4–5, читання та перевірка звіту:     ні
етап 6, рішення про закупівлю:             ні
етапи 7–8, перевірка та цілочисельність:   код, перевірено
етапи 9–11, транспортна задача:            текст, перероблено
------------------------------------------------------------------------------
----
Розв'язки транспортної задачі, знаходження закономірностей, створення тепл
ових карт. Також для деяких запитань з транспортної задачі використано ШІ
для перевірки та пояснення результатів.
```

Крок 13. Фінальна самоперевірка

In [119…

```
перевірки = {
    'ПІБ і варіант заповнено':            bool(ПІБ) and 1 <= ВАРІАНТ <= 3
    'Excel і Python дали однаковий план': abs(прибуток - EXCEL_ПРИБУТОК)
    'перевірку «ненульові = зв’язні» зроблено': ненульових == звязних,
    'дані транспортної задачі введено':   len(постачальники) == 3 and len
    'баланс запасів і попиту':            sum(запас) == sum(попит),
    'декларацію заповнено':               bool(ІНСТРУМЕНТИ_ШІ.strip()) an
```

```
}
for k, v in перевірки.items():
    print(('OK   ' if v else 'НЕ ОК') + '  ' + k)
if all(перевірки.values()):
    print('\nТехнічна частина виконана.')
    print('Переконайтеся, що заповнені ВСІ клітинки з відповідями,')
    print('і виконайте Kernel → Restart & Run All перед здачею.')
```

```
OK     ПІБ і варіант заповнено
OK     Excel і Python дали однаковий план
OK     перевірку «ненульові = зв’язні» зроблено
OK     дані транспортної задачі введено
OK     баланс запасів і попиту
OK     декларацію заповнено

Технічна частина виконана.
Переконайтеся, що заповнені ВСІ клітинки з відповідями,
і виконайте Kernel → Restart & Run All перед здачею.
```